

Detecting Prompt Smells in WildChat User Prompts with an LLM-Based Analyzer

Md. Tayabur Rahman Chowdhury · 3 October 2026 · github.com/tayabur-masud/prompt-smells-iit

1. Objective

A prompt smell is a recurring weakness in how a user phrases a request to a large language model (LLM), such as missing context or an unclear reference, that is likely to degrade the answer. The goal of this project was to build a configurable, fault-tolerant Python system that extracts English user prompts from the public WildChat dataset, classifies each prompt against ten predefined smell categories (plus Other) using an LLM, validates every verdict, and produces a JSON dataset of findings. The design was required to be provider-independent, resumable, and safe for hundreds of thousands of prompts.

2. Data

The input is the **allenai/WildChat** dataset [1, 2], file train-00003-of-00006.parquet (269 MB). Only the four required columns (conversation_id, model, timestamp, conversation) are read. A user message is kept only when its role is "user" and its own language label is "English"; assistant messages are never analyzed. For this study, the first 1,000 unique English prompts were exported to data/prompts_1000.parquet so that every later run analyzes exactly the same set.

Stage	Count	Note
Conversations in file	88,238	One Parquet shard of WildChat
User messages	189,258	role = user
English user messages	96,185	per-message language label
Unique English prompts	95,298	887 exact repeats dropped
Study sample	1,000	437 conversations, 2 Jul 2023 10:20–16:29 UTC, all gpt-3.5-turbo

3. System Design

The system is a staged pipeline in which each stage has one responsibility and communicates through small validated data objects:

Parquet → read columns → parse → filter (user, English) → LLM analysis → validate → records → JSON

Module	Responsibility	Design pattern / principle
pipeline.py	Orchestrates stages, bounded concurrency, resume, run summary	Pipes and Filters; Facade
dataset_loader.py	Download and cache dataset; streamed, column-projected reads	Iterator (lazy streaming)
conversation_parser.py	Role and language filtering; tolerant of malformed records	Filter; fail-soft
llm_client.py	OpenAI-compatible client, retries with backoff, rate limiter	Adapter; Decorator (retry)
prompt_analyzer.py	System prompt, JSON repair, Pydantic validation, category mapping	Single Responsibility; DTO validation
output_writer.py	Append-only checkpoint, failure log, atomic JSON output	Memento-style checkpoint
config.py	Layered settings: defaults ← YAML ← .env ← CLI; secret API key	Strategy (provider and input mode)

4. Method

- **Analyzer prompt.** The user prompt is wrapped in <prompt> tags and treated strictly as data. The model must report only evidence-based smells, must not treat length or complexity as a smell, and must answer with JSON only: {"smells": [{"type", "reason"}]}.
- **Validation.** Each answer is parsed, safely repaired if needed (code fences, trailing commas, reasoning tags), validated against a Pydantic schema, and mapped to a canonical category. Invalid answers are re-requested up to three times.
- **Output.** One record is written per detected smell; a prompt with no smell gets a single record with null smell fields. Files are UTF-8 with ensure_ascii=False.
- **Reliability.** Each prompt has a SHA-256 identifier; finished prompts are appended to a checkpoint so that --resume never re-sends them. Transient errors are retried with exponential backoff that honours the provider's suggested delay; a client-side limiter caps requests per minute; permanent errors (invalid key, exhausted credits, daily quota) stop the run immediately.
- **Testing.** 70 unit tests with a mocked LLM cover parsing, filtering, response repair, retries, checkpointing, resume, and output validation.

5. Execution

The first provider, OpenAI gpt-3.5-turbo, could not be used because the account had no credits; this exposed the need for the fail-fast handling of permanent errors. Because the client is provider-independent, switching to Google Gemini (gemini-3.5-flash-lite) required only three configuration values and no code change. Its free tier allows 15 requests per minute and about 500 per day, so the run was throttled to 14 requests per minute and completed over three resumed sessions, finishing with a second API key; no prompt was ever sent twice. Running against real data also revealed and fixed four defects: conversations delivered as NumPy arrays

were silently dropped, Gemini's retry delay was hidden in the response body, request logging flooded the console, and a safety-filtered response crashed the client.

6. Results

Of the 1,000 prompts, **999 were analyzed** and 1 was refused by Gemini's safety filter (logged as a failure, not reported as "no smell"). The output file, `output/prompt_smells.json`, contains 1,088 records and passed schema validation.

Outcome	Prompts	Share	Comment
At least one smell	826	82.7%	737 with one smell, 89 with two
No smell	173	17.3%	Clear, self-contained requests
Refused by safety filter	1	–	Excluded from output

Smell category	Prompts	% of 999	Typical example
Vague / Missing Context	563	56.4%	"hey there", "Can you create image ?"
Ambiguous References	207	20.7%	"it", "this" with no referent
Prompt Bloat / Convolved Prompt	45	4.5%	Rambling multi-part text
Format Ambiguity	36	3.6%	Output format left unclear
Overloaded Prompt	29	2.9%	Unrelated tasks combined
Irrelevant / Excessive Persona	14	1.4%	Role-play setup dominates
Conflicting Constraints	9	0.9%	Contradictory requirements
Unnecessary Repetition; Bias / Loaded Framing	5 + 5	0.5% each	–
Formality / Audience Mismatch; Other	1 + 1	0.1% each	–

7. Discussion and Limitations

- **Context-related smells dominate.** Vague / Missing Context and Ambiguous References account for most findings. Many WildChat messages are short follow-ups (e.g. "websockets without external api") that are clear within their conversation but were, as specified, judged in isolation; this inflates both categories.
- **Single automatic labeler.** All labels come from one LLM with no human-annotated gold standard, so counts reflect that model's judgement and could differ with another model.
- **Sample scope.** The 1,000 prompts cover a six-hour window on one day and a single source model, and WildChat's language labels are imperfect (some "English" prompts mix in Chinese). The results describe this sample, not the full dataset.
- **Scale.** The full shard (95,298 prompts) was not processed: at the free-tier quota it would take about 190 days. Batching about 20 prompts per request or a paid tier would make it feasible in days or hours.

8. Conclusion

The project delivered a working, tested, and provider-independent prompt smell detector and applied it to 1,000 real WildChat prompts. The outcome is a validated dataset showing that more than four in five prompts contain at least one smell, overwhelmingly a lack of context or an unclear reference. The layered design made it possible to change provider, input source, and rate limits through configuration alone, and the checkpoint design allowed a quota-limited run to finish across several sessions without repeated work. Recommended next steps are context-aware analysis that includes preceding turns, human validation of a labelled subset, and batched requests to process the full dataset.

References

- [1] Allen Institute for AI, "WildChat" dataset, Hugging Face. <https://huggingface.co/datasets/allenai/WildChat>
- [2] Data file used: <https://huggingface.co/datasets/allenai/WildChat/blob/main/data/train-00003-of-00006.parquet>
- [3] Project source code and results: <https://github.com/tayabur-masud/prompt-smells-iit>